

HumanEval Training Dynamics Analysis

RankAlign vs. TC methods — why Upper and Multi differ

Automated Analysis Report (in progress)

June 19, 2026

Abstract

This report mirrors the earlier RankAlign training-dynamics analysis (`analysis/report.tex`, membership/IFEval) but for the **HumanEval** task, across **gemma-4-31b** and **qwen-3.5-9b**, comparing the two HumanEval datasets **v2.1correct-upper** and **v2.1correct-multi**. It targets three questions: **(Q1)** why does RankAlign (s2) beat our TC methods (s4 self-TC, s7 neg-TC) on *multi*, while our methods win on *upper* (gemma-4)? **(Q2)** is typicality-correction (TC) training helping? **(Q3)** are the negative results on the test tables an *overfitting* effect, or are they already present on the train split? All generator metrics are typicality-corrected (tc) gen ROC AUC unless noted; each method is shown in its natural eval mode (RankAlign/s3 → Neg base; s4 → PMI base; s7 → Neg base), matching the convention in `docs/rerun_only_tables.tex`.

Status (2026-06-19): the TEST-set comparison (all settings, both models, both datasets) is complete. The train-set dynamics currently exist only for **gemma-4 s2/s4** (GROUP B). Train-set evals for **gemma s3/s7** and **all qwen settings** are running on mll (38-job `trs2` batch); sections depending on them are marked

TODO:

Contents

1	Q1: The Upper-vs-Multi Effect (TEST set)	2
2	Q3: Train vs. Test — Is It Overfitting?	2
3	Q2: Is TC Training Helping? (Focused TC comparison)	3
4	Generator–Validator Correlation (TEST)	3
5	Why does TC behave so differently? (mechanism)	4
5.1	Eval-time: the typicality correction does MORE work on multi	4
6	Q2 (cont.): Training Diagnostics — WandB	4
7	Diagnostics: Concordance / Correlation / Scatter	4
8	Conclusions (preliminary)	4

1 Q1: The Upper-vs-Multi Effect (TEST set)

Table 1 reports tc gen ROC AUC ($\times 100$) on the held-out test split for every setting, each in its natural eval mode. The key pattern your question rests on:

- **gemma-4 / upper**: our methods *win* — FLORA-Neg (s7) = 94.8 and FLORA-PMI (s4) = 90.7 both beat RankAlign (s2) = 88.2.
- **gemma-4 / multi**: RankAlign *wins* — s2 = 94.2 vs s7 = 92.0, s4 = 90.1.
- **qwen**: RankAlign wins on *both* datasets (multi especially, 96.5). The “ours-wins-on-upper” effect is **gemma-4-specific**.

Table 1: HumanEval TEST tc gen ROC AUC ($\times 100$). Each method in its natural eval mode (Base: self; SFT/s4/s13: PMI base; RankAlign/New+fsx/s7: Neg base). gemma-4 has no base eval.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (neg-TC)
gemma-4-31b	upper	—	82.9	88.2	87.9	90.7	94.8
gemma-4-31b	multi	—	87.9	94.2	94.0	90.1	92.0
qwen-3.5-9b	upper	69.1	87.9	90.2	92.6	83.2	88.9
qwen-3.5-9b	multi	69.0	89.4	96.5	96.7	89.8	87.9

TODO: Add the train-set counterpart of Table 1 once the **trs2** evals land (gemma s3/s7 + all qwen). Then

2 Q3: Train vs. Test — Is It Overfitting?

This is the central question. For the settings we already have on both splits (**gemma-4 s2 RankAlign** and **s4 FLORA-PMI**, epoch 2), Table 2 compares train-split and test-split tc gen ROC.

Table 2: gemma-4 train vs. test tc gen ROC ($\times 100$), epoch 2, each method in its natural eval mode. Train = --train split, N=50 stratified candidates/problem. Δ = train – test (large positive $\Rightarrow$ overfitting).

Data	Setting	Train	Test	Δ (train–test)
upper	RankAlign	83.4	88.2	-4.8
upper	FLORA-PMI (self-TC)	90.9	90.7	0.2
multi	RankAlign	94.6	94.2	0.4
multi	FLORA-PMI (self-TC)	99.0	90.1	8.9

Early finding (gemma-4, s2 vs s4). On **multi**, FLORA-PMI (s4) shows a large train $\rightarrow$ test drop (99.0 $\rightarrow$ 90.1, Δ = +8.9) while RankAlign generalizes almost perfectly (94.6 $\rightarrow$ 94.2, Δ = +0.4). On **upper**, s4 does *not* overfit (90.9 $\rightarrow$ 90.7, Δ = +0.2). This is consistent with the hypothesis that **our TC method overfits on multi** — it is best on the training candidates but loses the advantage on held-out problems, which is exactly where RankAlign overtakes it at test time. RankAlign’s small train $\rightarrow$ test gap suggests it learns a flatter, more transferable ranking on this task.

TODO: This only covers s2 vs s4. The method that *wins* upper is **s7 (neg-TC)**, whose train-set eval is in t

TODO: Add the qwen train-vs-test rows (s2/s3/s4/s7) once **trs2** lands — needed to confirm whether qwen

3 Q2: Is TC Training Helping? (Focused TC comparison)

The clean test of TC is s3 (New+fsx, *no* TC) vs s4 (adds self-TC) and s7 (adds neg-TC), evaluated in the matching TC mode (tc gen ROC, base-typ). Table 3 gives the deltas at test (epoch 2).

Table 3: TC effect at TEST: s3→s4 (self mode) and s3→s7 (neg mode), tc gen ROC ($\times 100$); parenthetical = Δ from adding TC.

Model	Data	s3 (self)	s4 (+self-TC)	s3 (neg)	s7 (+neg-TC)
gemma-4-31b	upper	87.5	90.7 (+3.2)	87.9	94.8 (+6.8)
gemma-4-31b	multi	90.9	90.1 (-0.7)	94.0	92.0 (-2.1)
qwen-3.5-9b	upper	86.8	83.2 (-3.6)	92.6	88.9 (-3.7)
qwen-3.5-9b	multi	92.3	89.8 (-2.4)	96.7	87.9 (-8.8)

Finding. TC training *helps only on gemma-4 / upper* (s4 +3.2, s7 +6.8 over the no-TC s3). Everywhere else it *hurts* at test: gemma multi (-0.7 , -2.1), and qwen on both datasets (down to -8.8 on qwen multi). This is the *opposite* of the old membership/IFEval report, where TC helped consistently — on HumanEval, the TC signal transfers only in the one regime (gemma upper) where our methods also win Table 1.

TODO: Add the train-side TC deltas once **trs2** lands. Hypothesis: TC’s train-set gain is large everywhere,

4 Generator–Validator Correlation (TEST)

Spearman (Table 4) and Pearson (Table 5) between generator and validator scores, epoch 2, each method in its natural eval mode (tc), mean $\pm$ SE over 82 problems.

Table 4: Spearman ρ (gen vs val), tc, $\times 100$.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	42.2 $\pm$ 2.9	54.7 $\pm$ 2.7	55.4 $\pm$ 2.4	66.6 $\pm$ 1.8	74.9 $\pm$ 1.5
gemma-4-31b	multi	—	62.2 $\pm$ 1.7	75.9 $\pm$ 1.2	68.9 $\pm$ 1.7	73.0 $\pm$ 1.4	79.3 $\pm$ 1.5
qwen-3.5-9b	upper	30.6 $\pm$ 2.5	50.5 $\pm$ 2.2	60.9 $\pm$ 2.0	64.0 $\pm$ 1.8	64.1 $\pm$ 1.7	66.8 $\pm$ 1.5
qwen-3.5-9b	multi	33.8 $\pm$ 2.2	63.6 $\pm$ 1.6	65.4 $\pm$ 1.5	71.6 $\pm$ 1.4	67.4 $\pm$ 1.6	66.8 $\pm$ 1.5

Table 5: Pearson r (gen vs val), tc, $\times 100$.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	31.8 $\pm$ 3.2	55.5 $\pm$ 2.8	60.2 $\pm$ 2.5	63.3 $\pm$ 1.8	75.8 $\pm$ 1.5
gemma-4-31b	multi	—	55.7 $\pm$ 2.3	70.7 $\pm$ 1.2	70.4 $\pm$ 1.6	69.0 $\pm$ 1.6	78.1 $\pm$ 1.5
qwen-3.5-9b	upper	26.5 $\pm$ 2.4	45.4 $\pm$ 2.3	69.5 $\pm$ 1.7	67.6 $\pm$ 1.8	58.9 $\pm$ 2.0	68.5 $\pm$ 1.5
qwen-3.5-9b	multi	27.1 $\pm$ 2.4	57.8 $\pm$ 1.9	70.1 $\pm$ 1.1	72.4 $\pm$ 1.4	66.0 $\pm$ 1.7	67.9 $\pm$ 1.5

TODO: Generator–validator **concordance** (ordering-agreement fraction) and the **scatter** / **score-delta** hi

5 Why does TC behave so differently? (mechanism)

5.1 Eval-time: the typicality correction does MORE work on multi

Table 6 decomposes the eval-time effect: **raw** gen ROC (uncorrected log-prob ranking) vs. **tc** gen ROC, and the *lift* from the correction.

Table 6: Eval-time raw vs. tc gen ROC ($\times 100$), epoch 2, TC eval mode. lift = tc - raw.

Model	Data	Setting	raw	tc	lift
gemma-4-31b	upper	New+fsx	91.5	87.9	-3.6
gemma-4-31b	upper	FLORA-PMI (self-TC)	85.9	90.7	+4.8
gemma-4-31b	upper	FLORA-Neg (neg-TC)	83.6	94.8	+11.2
gemma-4-31b	multi	New+fsx	87.2	94.0	+6.9
gemma-4-31b	multi	FLORA-PMI (self-TC)	74.2	90.1	+15.9
gemma-4-31b	multi	FLORA-Neg (neg-TC)	68.1	92.0	+23.9
qwen-3.5-9b	upper	New+fsx	81.3	92.6	+11.3
qwen-3.5-9b	upper	FLORA-PMI (self-TC)	69.0	83.2	+14.3
qwen-3.5-9b	upper	FLORA-Neg (neg-TC)	65.8	88.9	+23.1
qwen-3.5-9b	multi	New+fsx	80.0	96.7	+16.7
qwen-3.5-9b	multi	FLORA-PMI (self-TC)	58.3	89.8	+31.6
qwen-3.5-9b	multi	FLORA-Neg (neg-TC)	45.0	87.9	+42.9

Key reframing. On **multi**, the *raw* generator score barely ranks at all (gemma s7 = 68.1; qwen s7 = 45.0, i.e. *below chance*), whereas on *upper* it is much healthier (gemma s7 raw = 83.6). This is a direct consequence of the dataset construction: **multi**’s $\Delta\log P$ -selected stylizations make the *correct* code **atypical**, so raw log-prob is a misleading signal — and typicality correction is exactly what rescues it (lift +23.9 gemma multi, +42.9 qwen multi). So TC is *not* failing on multi at eval; it is doing *more* work there. The corrected (tc) scores end up similar across datasets (≈ 88 –95). The real question is therefore *not* “does TC help at eval” (it does, especially on multi) but “why, once everything leans on TC on multi, does RankAlign (s2) still edge our s7 (94.2 vs 92.0)?” — which is a **train**→**test generalization** question, addressed in Section 2 and (fully) once **trs2** lands.

TODO: Mechanism evidence still to pull: (i) WandB loss-component dynamics (Pref/NLL-G/NLL-V, score-

6 Q2 (cont.): Training Diagnostics — WandB

WandB logging was *online* for all HumanEval runs (project **rankalign**; run names **g4**-{cu,cm}-s{S} and **q35**-{cu,cm}-s{S}).

TODO: Pull and plot the loss curves (SFT vs RankAlign vs Ours; TC comparison) for gemma-4 and qwen d

7 Diagnostics: Concordance / Correlation / Scatter

TODO: Compute generator–validator concordance, Spearman, and Pearson (epoch 2) and the gen-vs-val sca

8 Conclusions (preliminary)

1. **The upper-vs-multi flip is real and gemma-4-specific:** our TC methods (s7, s4) beat RankAlign on gemma-4 *upper* but lose on *multi*; on qwen RankAlign wins both.

2. **Preliminary overfitting signal:** gemma-4 FLORA-PMI overfits on multi ($\Delta = +8.9$ train–test) but not on upper ($\Delta = +0.2$), while RankAlign generalizes on both. This is consistent with the multi loss being an overfitting effect.

TODO: Finalize conclusions once s7/s3/qwen train-set evals + WandB + concordance land. Open questions

Reproducibility. Tables built by `analysis/scripts/build_he_report_tables.py` from `docs/he_harvest_2026-06-19` (test) and `docs/he_trainset_2026-06-19/humaneval_TRAIN_metrics_aggregated.csv` (train, gemma s2/s4). trs2 train-set batch: `scripts/ml1_he_trainset_eval.v2.sbatch + launch_he_trainset_extra`. model origins in `docs/he_trainset_eval_model_origins.md`.